Wireshark

4/18/26 6:32 PM

Identify Common Cyber Network Attacks with Wireshark

Core Concept: Wireshark is a **packet-level microscope**. Use it **after** you have a lead from IDS alerts, firewall logs, or server event logs. Do not start by randomly capturing everything — that wastes time and misses key traffic.
When to use Wireshark:
- After an IDS alert (Suricata/Snort/Zeek).
- After firewall or server log events show suspicious activity.
- To map activity to **MITRE ATT&CK** (Reconnaissance → Initial Access → Execution → etc.) or **Cyber Kill Chain**.

Best Practice:
- Capture 24/7 at key points (internet ingress, core server links).
- Always start from a specific lead (IP, port, time, event) instead of full capture.

Module 3: Analyzing Port Scans and Enumeration Methods

Core Concepts: Attackers first discover hosts and open ports before exploiting them. Common methods: ARP scans, ICMP ping sweeps, TCP SYN scans, OS fingerprinting, HTTP path enumeration.

Lab 1 – Detecting Network Discovery Scans

- Filter: 'arp'.
- Look for one source MAC sending ARP requests for many IPs in the subnet (one-to-many pattern).
- Right-click ARP request → Prepare as Filter → Selected → change opcode to '2' for replies.
- ARP replies reveal exactly which hosts are active on the network — the first step in attacker reconnaissance.

Lab 2 – Identifying Port Scans (Part 1 & 2)

- Filter: 'tcp.flags.syn == 1 && tcp.flags.ack == 0' (SYN-only packets).
- Go to Statistics → Conversations → TCP tab → sort by Address B or Port B.
- Look for one source IP sending SYNs to many different ports/IPs.
- To see open ports: 'tcp.flags.syn == 1 && tcp.flags.ack == 1'.
- Save filter under TCP → Open Ports.
- SYN scans are the most common because they are fast and do not complete the handshake, making them harder for basic firewalls to log.

Lab 3 – Analyzing Malware for Network and Port Scans (Part 1 & 2)

- Filter: 'arp' → find source doing ARP sweep across subnet.
- Filter: 'icmp' → see ping sweep.
- Filter: 'tcp.flags.syn == 1 && tcp.flags.ack == 0' → see TCP SYN activity after discovery.
- Use Statistics → Conversations → IPv4 and TCP to see one-to-many patterns.
- Malware often performs the exact same discovery scans as manual tools like Nmap, making the behavior easy to spot once you know what to filter.

Lab 4 – Detecting OS Fingerprinting (Part 1 & 2)

- Filter: '!arp' to remove noise.
- Look for many crafted TCP SYNs from one source with unusual TCP options or TTL values.
- Add column: Right-click IP → Time to Live → Apply as Column.
- Look for TTL values jumping around (30–50 range is suspicious locally).
- Filter for Xmas scan: 'tcp.flags == 0x029' (URG+PSH+FIN set — "lights up like a Christmas tree").
- Filter for Null scan: 'tcp.flags == 0x000' (no flags set).
- Xmas and Null scans send malformed packets to see how the target OS stack responds, helping identify the OS version.

Lab 5 – Analyzing HTTP Path Enumeration

- Filter: 'http.request.method == "GET"'.
- Look for many sequential or dictionary-like GETs to different paths.
- Look for high number of 404 Not Found responses.
- Check User-Agent column for 'gobuster' (common enumeration tool).
- Gobuster is a popular open-source tool attackers use to brute-force hidden directories and files on web servers.

Module 4: Analyzing Common Attack Signatures of Suspect Traffic

Lab 6 – Analyzing TCP SYN Attacks

- Filter: 'tcp.flags.syn == 1 && tcp.flags.ack == 0'.
- Go to Statistics → Conversations → TCP → sort by Address B or Port B.
- Look for many SYNs from many sources to one target.
- Filter for responses: 'tcp.flags.syn == 1 && tcp.flags.ack == 1 && ip.dst == <target>'.
- SYN floods overwhelm the target's resources because it must allocate memory for every half-open connection.

Lab 7 – Spotting Suspect Country Codes with GeoIP

- Filter: 'ip.geoip.country == "Russia"' (or China, etc.).
- Go to Statistics → Endpoints → look at GeoIP columns.
- Filter for outbound: 'ip.geoip.country == "Russia" && ip.src == <internal range>'.

Lab 8 – Filtering for Unusual Domain Name Lookups

- Filter: 'dns.qry.name contains "hackersden"'.
- Use regex for multiple countries: 'dns.qry.name matches "(ru|cn|jp|uk)"'.
- Unusual country-code domains are a quick indicator of reconnaissance or C2 domains attackers commonly register.

Lab 9 – Analyzing HTTP Traffic and Unencrypted File Transfers

- Filter: 'http.request.method == "GET" || http.request.method == "POST"'.
- Look for .exe, .bin, .php in Request URI.
- Go to File → Export Objects → HTTP to extract suspicious files.
- Filter for executables: 'http.request.uri matches "(\.exe|\.bin|\.php|\.zip)"'.
- Attackers disguise executables as .png or other harmless extensions to bypass basic filters.

Lab 10 – Analysis of a Brute Force Attack

- Filter: 'ftp' or 'tcp.port == 21'.
- Follow TCP Stream on login attempts.
- Look for many "530 Login incorrect" responses followed by one "230 Login successful".
- Response code 230 means successful FTP login — the moment the attacker gains access.

Module 5: Identifying Common Malware Behavior

Lab 11 – Malware Analysis with Wireshark (Part 1 & 2)

- Filter: 'http.request || http.response'.
- Look for POSTs to unusual countries.
- Follow TCP Stream on POSTs → look for stolen data (usernames, passwords, PC name).
- Go to File → Export Objects → HTTP → look for .exe disguised as .png.
- Malware often hides executables inside image-like filenames (e.g., cursor.png) to blend with normal web traffic.

Module 6: Identify Shell, Reverse Shell, Botnet, and DDoS Attack Traffic

Lab 12 – Analyzing Reverse Shell Behavior

- Filter: 'tcp.port == 1337' (common reverse shell port).
- Follow TCP Stream → look for command-like traffic (ls, whoami, cd).
- Reverse shell works by planting a PHP script on a vulnerable web server that forces the server to connect back to the attacker on the chosen port.

Lab 13 – Identifying Botnet Traffic (Part 1 & 2)

- Filter: http.request || http.response.
- Look for many GETs/POSTs from one client to unusual countries.
- Follow TCP Stream on POSTs → look for process lists, PC name, credentials.
- Use tcp.analysis.flags to spot retransmissions and unusual TCP behavior that stands out from normal traffic.
- Botnet C2 traffic often uses HTTP POSTs to phone home with system information while blending with normal web traffic.

Lab 14 – Analyzing Data Exfiltration with Wireshark

- Filter: ftp or tcp.port == 21.
- Follow TCP Stream on login → look for "STOR" (upload) commands.
- Look for large outbound transfers after malware download.
- After downloading the .exe, the client waits ~30 minutes then uploads stolen data (usernames, passwords, banking info) via FTP STOR commands in cleartext.

Investigate Cyber Network Attacks with Wireshark LAB:

Analyzing Command & Control (C2) Infrastructure

Concept: C2 traffic is the ongoing communication between a compromised host and the attacker's server for commands, status checks, and data exfil. It often tries to blend in with normal traffic.
What to look for:
- Beaconing pattern: packets sent at very regular intervals (e.g., every 5.03 seconds).
- Consistent small packet sizes across many sessions.
- Communication on port 80 (HTTP) with encrypted payload (random bytes in Packet Bytes pane, not readable HTTP).
- No normal HTTP requests/responses even though port 80 is used.

Steps in Wireshark:
1. Open Conversations (Statistics → Conversations → TCP tab).
2. Check **Rel. Start** column for regular timing between sessions.
3. Check packet **Length** column for consistent small sizes.
4. Select a session → look in **Packet Bytes** pane for random/encrypted data (no readable HTTP headers).
5. Compare with normal HTTP traffic (tcp.stream eq X) to see missing HTTP layer.

Why it matters: Regular timing + small consistent packets + encrypted data on port 80 is a strong indicator of beaconing C2, even when it mimics web traffic.

Investigating SSH Brute Force Attack

Concept: Brute force attempts many username/password combinations rapidly against SSH (port 22).
What to look for:
- Many short TCP sessions from the same source to port 22.
- Rapid repeated connection attempts (visible in Flow Graph or Conversations).
- Consistent short duration and packet counts per failed attempt.
- No successful key exchange or very quick resets.

Steps in Wireshark:
1. Filter: tcp.port == 22.
2. Open **Statistics → Conversations → TCP** → sort by Address or Port.
3. Open **Statistics → Flow Graph** to visualize repeated short connection attempts.
4. Look at **Time** column for attempts happening in ~0.3 seconds intervals.

Why it matters: Legitimate users rarely attempt logins this quickly and repeatedly. The pattern stands out clearly in Conversations and Flow Graph.

Investigating C2 from a Metasploit Server (Meterpreter)

Concept: Meterpreter C2 often uses encrypted channels that look like normal traffic but show unusual session behavior.
What to look for:
- Very small packet lengths and short session durations.
- Communication on port 80 with no actual HTTP protocol data (no GET/POST headers).
- Low percentage of HTTP in Protocol Hierarchy despite using port 80.
- Consistent small back-and-forth packets (beaconing).

Steps in Wireshark:
1. Open **Statistics → Conversations → TCP** → look for sessions with tiny byte counts.
2. Filter: tcp.stream eq 3 (or the suspicious stream number).
3. Check **Protocol Hierarchy** (Statistics → Protocol Hierarchy) → HTTP should be near 0% for that stream.
4. Look in Packet Details for missing HTTP layer even on port 80 traffic.

Why it matters: Attackers use port 80 to blend in, but the lack of real HTTP and the regular small-packet pattern reveals C2 activity.